Отчёт по индивидуальному заданию № 3

Национальный исследовательский университет «Высшая школа экономики»

Факультет компьютерных наук, Департамент программной инженерии

Дисциплина: «Архитектура компьютера и операционные системы»

Вариант 24: «Танковые бои». Взаимодействующие процессы

Выполнил: Адаменко Мирослав Александрович

Группа: БПИ-252

Москва, 2026

1. Условие и принятая модель

По условию варианта 24 на клеточном поле с препятствиями сражаются две стороны. Танки наблюдают цели, выбирают движения и стреляют; решения сторон формируются до раскрытия результатов и применяются по общей фазовой схеме. Нужно отображать наблюдения, намерения, конфликты и перемещения, выбор цели, выстрелы, попадания, повреждения, уничтожения и состояние поля. Бой заканчивается уничтожением одной или обеих сторон, по пределу тактов либо по прерыванию; формируется статистика.

Условие предусматривает размеры и препятствия поля, число, позиции и характеристики танков, фазы, стратегии, тип и параметры стрельбы, правила столкновений и уничтоженных машин и предел тактов. В реализации поле имеет 3–12 строк и 1–12 столбцов; препятствия генерируются по заданной вероятности, состав команд задаётся числом или количеством танков четырёх типов. Начальные позиции, характеристики, стратегия и порядок фаз определены моделью; они не задаются произвольно. Стрельба ведётся прямой наводкой по видимым целям, попадание применяется в фазе стрельбы. Уничтоженные танки удаляются с поля; конфликт перемещений разрешается с учётом массы. Координатор может назначить цель сосредоточенного огня; ключ --no-focus отключает фокус. Зерно генератора позволяет воспроизвести случайные решения.

2. Сущности и процессы

Танк представлен отдельным процессом: он получает снимок состояния и вычисляет действия в соответствующей фазе. Два процесса координаторов обобщают наблюдения своих команд и определяют цель фокуса. Арбитр (родительский процесс в вариантах с fork(), независимо запущенный процесс в остальных): выдаёт задания, собирает ответы, изменяет состояние мира, печатает события и статистику, управляет временем жизни и завершением участников.

Поле, препятствия, выстрел и журнал не имеют самостоятельного поведения: это данные, управляемые арбитром. Отдельный процесс для каждого объекта поля или каждого мгновенного выстрела не нужен. Всего используется 1 + N_A + N_B + 2 процесса: по одному на танк, два на координаторов и один на арбитра. По умолчанию их 11, максимально — 15.

3. Порядок взаимодействия и инварианты

Арбитр организует такт фазами: наблюдение танками, обработка разведданных координаторами, формирование решений танками, применение выстрелов и повреждений, разрешение перемещений и отображение состояния. Участники получают снимок данных и возвращают результат; мутации общего мира выполняет арбитр. Переход к следующей фазе выполняется после сбора соответствующих ответов.

Такое разделение предотвращает изменение мира отдельными танками во время принятия решений. На поле два действующих танка не должны занимать одну

позицию; конфликтующие перемещения разрешаются однозначно. Прямой выстрел возможен только при наличии линии видимости. Уничтоженный танк больше не участвует в бою. Задания и результаты содержат данные, пригодные для межпроцессной передачи, а не указатели на динамическую память другого процесса.

4. Варианты IPC

Реализованы четыре запуска на общем коде модели и общем интерфейсе обмена. В tanks_unnamed родитель создаёт дочерние процессы через fork(), данные передаются через разделяемую память POSIX, а ожидание обеспечивают неименованные семафоры в этой памяти с pshared=1. В tanks_named также используются fork() и разделяемая память, но семафоры именованные.

В tanks_independent арбитр и работники запускаются независимо; связь организована через POSIX shm и именованные семафоры. Арбитр создаёт сессию, печатает команды запуска работников и ожидает их готовности. В tanks_queue процессы также независимы, но полезные данные передаются в очередях сообщений POSIX, а синхронизация использует именованные семафоры. Небольшая разделяемая область служит для метаданных и регистрации подключений, не для передачи модели боя.

Для каждого работника организованы направления запросов и ответов. Имена объектов привязаны к имени сессии. Создание использует эксклюзивное открытие; при частичной ошибке созданные ресурсы убираются. Закрытие дескрипторов процесса отдельно от удаления именованных объектов: удаляет их создатель. Ожидания ограничены по времени; блокировка регистрации ограничена пятью секундами. Ошибка обмена передаётся вызывающей стороне.

5. Интерфейс и запуск

Программа работает в консоли. --help выводит параметры; --batch запускает без интерактивных вопросов, --plain выводит журнал построчно, --seed задаёт зерно генератора, --delay — паузу между тактами в миллисекундах, --max-ticks — предел тактов, --log — файл журнала. Пример запуска первых двух вариантов:

```
make
./tanks_unnamed --batch --plain --seed 1 --delay 0 --max-ticks 10
./tanks_named --batch --plain --seed 1 --delay 0 --max-ticks 10
```

Для независимых вариантов аналогично запускается tanks_independent либо tanks_queue. Арбитр печатает команды tanks_worker с именем сессии, ролью и номером; их следует выполнить в другой оболочке, добавив к каждой команде &; затем команда wait дождётся завершения. Арбитр ожидает готовности каждого работника до 30 секунд, после чего начинает бой или сообщает об ошибке. При достижении предела или уничтожении стороны он завершает участников и освобождает ресурсы. SIGINT и SIGTERM позволяют прервать выполнение; коды 130 и 143 соответствуют этим сигналам.

6. Проверки и результаты

Итоговый запуск make test: 58 групп проверок пройдены, ошибок нет. Проверены четыре схемы, граничные размеры и составы, seed, завершение и сигналы, неполный запуск, таймаут блокировки, гибель участника и ошибки записи. Отдельно проверены ввод по умолчанию и вручную, повтор ввода после ошибки, окончание ввода, прерывание на настройке, ANSI-вывод и восстановление курсора при штатном и прерванном завершении. Проверка IPC охватила три режима обмена, таймаут, недопустимый размер и конфликты регистрации. Выборочный запуск с ASan и UBSan для двух вариантов с fork и варианта очередей не выявил ошибок; это не проверка всех сценариев.

Пример результата при поле 10 × 10, по четыре танка и seed 27: обе команды уничтожены, ничья на 145-м такте; разрешено шесть конфликтов перемещения. Команда A выполнила 15 выстрелов и нанесла 760 урона, команда B — 16 выстрелов и 900 урона. Нормализованные журналы четырёх вариантов для этого набора совпали с эталоном ИД32; эталонный хеш зависит от glibc.

Соответствие требованиям:

Требование	Реализация
Процессы участников	Танк, два координатора, арбитр (§ 2)
Синхронизация и инварианты	Фазы и один владелец поля (§ 3)
Четыре схемы взаимодействия	Память, семафоры, очереди (§ 4)
Запуск, вывод, завершение	Консоль, журнал, сигналы (§ 5)

7. Ограничения и обоснование решений

Программа является имитацией, а не физически точной моделью боя. Часть параметров условия — например, начальные позиции, характеристики танков, стратегия и фазы — закреплена в модели. Расположение препятствий задаётся вероятностью, а не произвольной картой. Прямая наводка и удаление уничтоженных машин выбраны из разрешённых условием вариантов.

Модель требует Linux с поддержкой POSIX shared memory, семафоров и очередей сообщений; максимальный размер сообщения очереди должен быть не менее 8192 байт. Таймаут ожидания готовности каждого независимого работника — 30 секунд. Сигналы и обычные ошибки предусматривают очистку ресурсов, но принудительное завершение SIGKILL самого владельца не позволяет ему выполнить очистку. Результаты тестирования ограничены проверенными входами и не доказывают отсутствия всех возможных гонок или ошибок системы.

В разделяемую память передаются структуры без указателей, поскольку адреса динамических буферов процесса недействительны в другом процессе. Арбитр один изменяет состояние поля; обмен задачами и результатами отделён от применения решений. Это упрощает соблюдение фазовых инвариантов и позволяет использовать четыре схемы IPC с одной моделью.